

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА
Институт искусственного интеллекта
Кафедра проблем управления

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине **Основы программирования систем управления**

Тема лабораторной работы: «Отладка программного обеспечения роботехнических систем с использованием виртуального моделирования»

Студенты группы: КРБО-01-23 Корнев К. А. _____

Мино П. С. _____

Ярошенко А. В. _____

Преподаватель: ст. преподаватель Морозов А. А. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

Цель работы

Получение навыков моделирования объекта управления в промышленных системах автоматического управления и создание функциональных блоков.

Задание

Необходимо разработать проект в среде Automation Studio 4.2, реализующий виртуальную систему управления, включающую:

- модель объекта управления (двигателя постоянного тока) с передаточной функцией
$$W = \frac{1}{k_e(T_m s + 1)};$$
- ПИ-регулятор;
- сумматор и обратную связь.

Для выполнения задания необходимо изучить типы и правила инициализации переменных, механизм использования условных операторов, принцип циклических программ, а также освоить создание функциональных блоков в среде Automation Studio.

Порядок выполнения работы

1. Создать проект и добавить в него перечень объектов для реализации требуемых моделей;
    1.1. Создать новый проект в среде Automation Studio без конфигурации оборудования;
    1.2. Создать в проекте объекты:
        1.2.1. ANSI C Program;
        1.2.2. ANSI C Library «MotorControl».
    1.3. В библиотеке создать 3 функциональных блока и дать им имена:
        1.3.1. «FB_Motor», в котором далее реализуем модель двигателя постоянного тока;

1.3.2. «FB_Regulator», выполняющий роль модели ПИ-регулятора;

1.3.3. «FB_Integrator», представляющим из себя модель интегрирующего звена.

Ход работы

В начале работы был создан новый проект в среде Automation Studio 4.2 без конфигурации оборудования. Затем в проект были добавлены объекты: ANSI C Program и ANSI C Library «MotorControl». В библиотеке были созданы три функциональных блока, реализующих моделирование системы управления: `FB_Integrator`, `FB_Motor` и `FB_Regulator`. Основная программа, объединяющая объект и регулятор в систему управления, реализована в файле `Main.c`, содержащем подпрограмму инициализации `Init.c` и основной цикл `Cyclic.c`.

Функциональный блок `FB_Integrator` представляет собой модель интегрирующего звена. Назначение блока и его параметры приведены в таблице 1. Логика работы интегратора заключается в накоплении суммы разностей входного и выходного значений в соответствии с шагом расчета. В соответствии с исходным кодом, выходная переменная вычисляется как `out += in * dt`, при этом в начале функции предусмотрена проверка указателя на нулевой адрес. Структурная схема интегратора приведена на рисунке 1.

Таблица 1 — Параметры функционального блока `FB_Integrator`

Конфигурация	**Имя**	**Описание**
вход	`in`	вход интегрирующего звена
выход	`out`	выход интегрирующего звена
внутреннее состояние	`dt`	шаг расчета [c]

Далее был разработан функциональный блок `FB_Motor`, моделирующий двигатель постоянного тока. В структуру блока были введены поля, описанные в таблице 2. Программный код блока использует приведенные поля в соответствии с их описанием. Расчет значения на выходе блока происходит в соответствии со схемой ДПТ или путем вывода из передаточной функции объекта разностного уравнения методом Z-преобразования. Согласно исходному коду, модель реализует дифференциальное уравнение первого порядка относительно

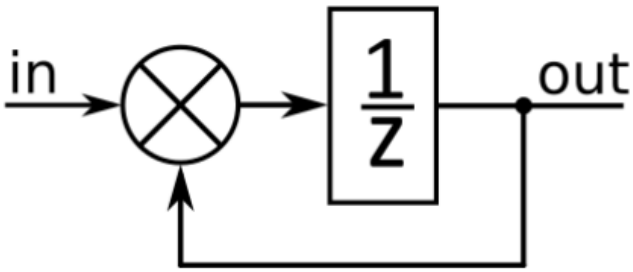


Рис. 1 — Структурная схема интегратора

частоты вращения:

$$\frac{d\omega}{dt} = \frac{u/k_e - \omega}{T_m},$$

где величина u/k_e играет роль установившегося значения частоты вращения. Полученное приращение подается на вход интегратора, выход которого соответствует текущей частоте вращения ω. Положение φ вычисляется интегрированием частоты вращения: $\varphi \mathrel{+}= \omega \cdot dt$. Значения параметров $k_e = 0.05$, $T_m = 0.05$ с, шаг расчета $dt = 0.01$ с заданы в подпрограмме инициализации. Структура объекта управления представлена на рисунке 2.

Таблица 2 — Параметры функционального блока `FB_Motor`

Конфигурация	**Имя**	**Описание**
вход	`u`	входное напряжение [В]
выход	`w`	частота вращения [об/мин]
выход	`phi`	положение [рад]
внутреннее состояние	`integrator`	интегратор
внутреннее состояние	`Tm`	электромеханическая постоянная времени [с]
внутреннее состояние	`ke`	постоянная ЭДС двигателя

Затем был создан функциональный блок `FB_Regulator`, выполняющий роль модели ПИ-регулятора. В блок были внесены поля, приведенные в таблице 3. Программный код использует данные поля в соответствии с их описанием. Расчет значения на выходе блока происходит в соответствии со схемой ПИ-регулятора, приведенной на рисунке 3. Схема противонакопления используется для ограничения напряжения, поступающего на двигатель, в целях обеспечения стабильности его работы. Также в схеме присутствуют два ограничителя напряжения: после П-звена и выходного напряжения. Величина ограничения задается

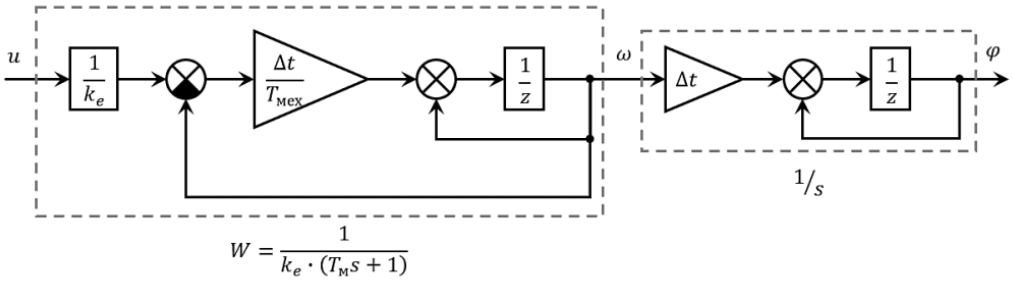


Рис. 2 — Структура объекта управления

с помощью внутреннего состояния `max_abs_value`. Согласно исходному коду, пропорциональная часть вычисляется как $u_p = k_p \cdot e$ и ограничивается по абсолютной величине. Интегральная часть формируется интегратором, на вход которого подается $k_i \cdot dt \cdot e + i_{yOLD}$, где i_{yOLD} — сигнал противонакопления. Выходной сигнал регулятора равен сумме $u_p + u_i$ с последующим ограничением. Сигнал противонакопления вычисляется как разность между ограниченным и неограниченным выходным сигналом и сохраняется для следующего шага. Расчет коэффициентов регулятора производится с помощью метода обратной задачи динамики (ОЗД). Принято $T = 0.02$ с. Формулы для коэффициентов:

$$k_p = \frac{k_e \cdot T_m}{T}; \quad k_i = \frac{k_e}{T}. \tag{1}$$

В соответствии с исходным кодом, $k_e = 0.05$, $T_m = 0.05$ с, $T = 0.02$ с, что дает $k_p = 0.125$ и $k_i = 2.5$. Граница ограничения $U_{max} = 24.0$ В, шаг расчета $dt = 0.01$ с.

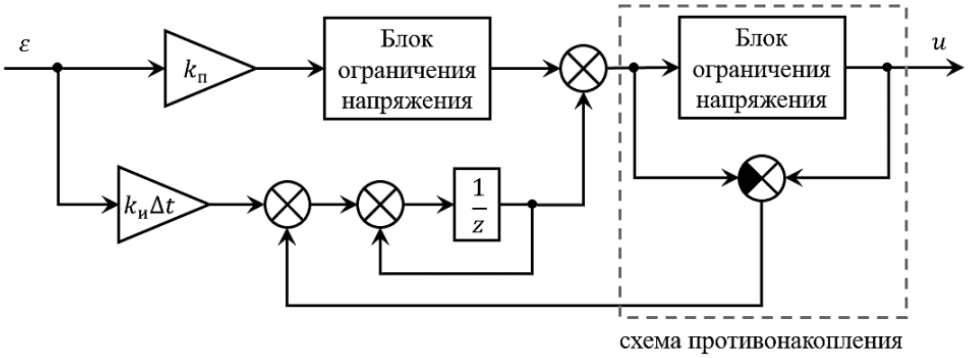


Рис. 3 — Структура ПИ-регулятора

После создания всех функциональных блоков была разработана основная программа `Main.c`, объединяющая объект и регулятор в систему управления. В

Таблица 3 — Параметры функционального блока FB_Regulator

Конфигурация	**Имя**	**Описание**
вход	e	рассогласование между задающим воздействием и реальной скоростью вращения вала ДПТ [об/мин]
выход	u	напряжение, подаваемое на вход ДПТ [В]
внутреннее состояние	k_p	пропорциональный коэффициент регулятора
внутреннее состояние	k_i	интегральный коэффициент регулятора
внутреннее состояние	integrator	интегратор
внутреннее состояние	iyOLD	хранение предыдущего значения схемы противонакопления
внутреннее состояние	max_abs_value	граница блока ограничения [В]
внутреннее состояние	dt	шаг расчета [с]

переменных основной программы были созданы поля, приведенные в таблице 4. В подпрограмме инициализации ProgramInit заданы все постоянные (коэффициенты регуляторов, постоянные времени, граничные значения и шаги расчета) созданных объектов, а также обнулены переменные Counter, Speed и Enable. В основном цикле ProgramCyclic реализована система управления, имеющая на вход значение переменной Speed и активирующаяся при наличии логической «1» в переменной Enable.

В программе инициализируются два экземпляра двигателя (fb_motor и fb_motor_2) с одинаковыми параметрами. Первый двигатель охвачен обратной связью через регулятор, второй используется для сравнения и работает без регулятора. В основном цикле формируется ступенчатое задающее воздействие с помощью переменной-счетчика Counter: при превышении порога 100 циклов уставка Speed принимает значение 90.0 об/мин, а сигнал Enable устанавливается в «1»; при превышении порога 250 циклов уставка сбрасывается в 0.0. Входное напряжение второго двигателя формируется как $u = \text{Enable} \cdot \text{Speed} \cdot k_e$, то есть реализуется разомкнутое управление, пропорциональное заданию. При включенном сигнале Enable для первого двигателя рассчитывается ошибка как разность между уставкой и текущей скоростью вращения, затем вызывается

регулятор, и его выход подается на вход двигателя. В противном случае напряжение на первом двигателе равно нулю, и двигатель затормаживается. Оба двигателя обсчитываются вызовом соответствующих функциональных блоков.

Таблица 4 — Переменные основной программы

Имя	**Тип данных**	**Описание**
`fb_regulator`	`FB_Regulator`	регулятор
`fb_motor`	`FB_Motor`	двигатель с регулятором
`fb_motor_2`	`FB_Motor`	двигатель без регулятора
`Speed`	`REAL`	уставка по скорости
`Enable`	`BOOL`	разрешение работы
`Counter`	`INT`	счетчик для формирования ступенчатого воздействия
`Tzh`	`REAL`	желаемая постоянная времени замкнутой системы, используемая при расчете коэффициентов регулятора

Для проверки работоспособности системы управления был проведен эксперимент. На систему было подано ступенчатое воздействие. В качестве задающего воздействия использовалась переменная `Speed`, график которой приведен на нижнем графике рисунка 4. Величина уставки составляла 90.0 об/мин. Ступенчатое воздействие подавалось периодически: сигнал изменялся с 0 до 90.0 и обратно с интервалом примерно 1.6 секунды.

График переходного процесса объекта управления без регулятора (`fb_motor_2.w`) приведен на верхнем графике. Как видно из графика, скорость вращения двигателя плавно нарастает и достигает установившегося значения 90.0 об/мин примерно за 0.3–0.4 секунды, что соответствует апериодическому звену первого порядка с постоянной времени $T_m = 0.05$ с. При снятии управляющего воздействия скорость также плавно снижается до нуля с той же постоянной времени.

График переходного процесса объекта управления с регулятором (`fb_motor_.w`) приведен на среднем графике. В этом случае скорость вращения достигает заданного значения 90.0 об/мин значительно быстрее — практически мгновенно (в пределах одного-двух циклов опроса), что свидетельствует о высокой эффективности настроенного ПИ-регулятора с коэффициентами

$k_p = 0.125$ и $k_i = 2.5$. На графике видны незначительные колебания в начальный момент, связанные с дискретностью системы управления и настройками коэффициентов, однако перерегулирование отсутствует, и выход на установившееся значение происходит без статической ошибки.

Сравнивая графики, можно сделать вывод, что применение ПИ-регулятора позволило существенно сократить время переходного процесса и повысить точность отработки задающего воздействия. Графики приведены на рисунке 4.

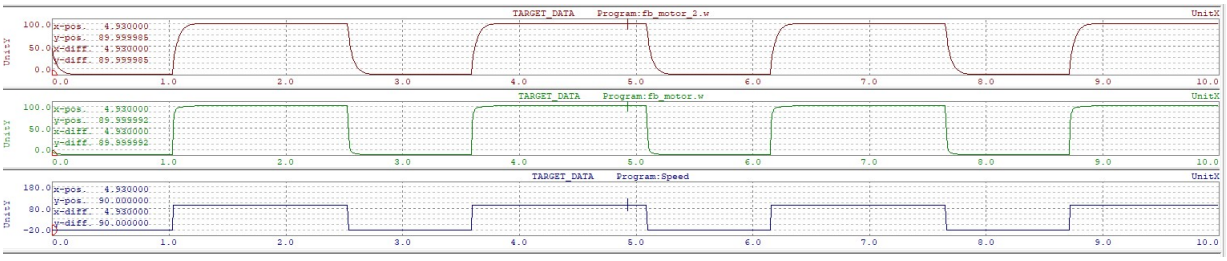


Рис. 4 — График уставки, скорости вращения двигателя без регулятора и скорости вращения двигателя с настроенным регулятором

Выводы

В ходе выполнения лабораторной работы были изучены базовые конструкции языка Си (ANSI C) и освоены принципы разработки программного обеспечения в среде Automation Studio 4.2 на основе функциональных блоков.

Были разработаны три функциональных блока: `FB_Integrator`, реализующий модель интегрирующего звена; `FB_Motor`, моделирующий двигатель постоянного тока в соответствии с его передаточной функцией; `FB_Regulator`, реализующий ПИ-закон регулирования со схемой противонакопления и ограничением выходного напряжения. В результате объединения данных блоков в основной программе `Main.c` была получена замкнутая система автоматического управления с обратной связью по скорости вращения.

Экспериментально подтверждена работоспособность разработанной системы. Сравнение переходных процессов двигателя без регулятора и с регулятором показало, что применение ПИ-регулятора с коэффициентами $k_p = 0.125$ и $k_i = 2.5$, рассчитанными методом обратной задачи динамики для значений $k_e = 0.05$, $T_m = 0.05$ с и $T = 0.02$ с, позволяет существенно сократить время переходного процесса и повысить точность отработки ступенчатого задающего воздействия. Двигатель без регулятора отрабатывает уставку 90.0 об/мин при-

мерно за 0.3–0.4 с, тогда как в замкнутой системе выход на заданное значение происходит практически мгновенно и без статической ошибки.

Таким образом, цель работы достигнута: получены практические навыки моделирования объекта управления, создания функциональных блоков на языке Си и настройки регулятора в среде Automation Studio.

Приложение А

Листинг исходного программного кода файла FB_Itegrator.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
	extern "C"
	{
#endif
	#include "Library.h"
#ifdef __cplusplus
	};
#endif
/* TODO: Add your comment here */
void FB_Integrator(struct FB_Integrator* inst)
{
	if (inst==0)
		return;
	inst->out += inst->in*inst->dt;
}
```

Приложение Б

Листинг исходного программного кода файла FB_Motor.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
	extern "C"
	{
#endif
	#include "Library.h"
#ifdef __cplusplus
	};
#endif
/* TODO: Add your comment here */
void FB_Motor(struct FB_Motor* inst)
{
	if (inst==0)
		return ;
	REAL u_norm = inst->u / inst->ke;
	REAL din = (u_norm - inst->integrator.out) / inst->Tm;

	inst->integrator.in = din;
	FB_Integrator(&inst->integrator);
	inst->w = inst->integrator.out;

	inst->phi += inst->w * inst->dt;
}
```

Приложение В

Листинг исходного программного кода файла FB_Regulator.c

```
#include <bur/plctypes.h>
#ifdef __cplusplus
	extern "C"
	{
#endif
	#include "Library.h"
#ifdef __cplusplus
	};
#endif
/* TODO: Add your comment here */
void FB_Regulator(struct FB_Regulator* inst)
{

	REAL up;
	REAL ui;
	REAL u_raw;
	REAL u_lim;

	if (inst->dt <= 0.0f) inst->dt = 0.01f;
	if (inst->max_abs_value <= 0.0f) inst->max_abs_value = 24.0f;

	up = inst->k_p * inst->e;

	if (up >  inst->max_abs_value) up =  inst->max_abs_value;
	if (up < -inst->max_abs_value) up = -inst->max_abs_value;

	inst->integrator.dt = 1.0f;
	inst->integrator.in = inst->k_i * inst->dt * inst->e + inst->iyOLD;
	FB_Integrator(&inst->integrator);
	ui = inst->integrator.out;

	u_raw = up + ui;

	u_lim = u_raw;
	if (u_lim >  inst->max_abs_value) u_lim =  inst->max_abs_value;
	if (u_lim < -inst->max_abs_value) u_lim = -inst->max_abs_value;

	inst->iyOLD = u_lim - u_raw;

	inst->u = u_lim;
}
```

Приложение Г

Листинг исходного программного кода файла Main.c

```
#include <bur/plctypes.h>
#include <Library.h>

#ifdef _DEFAULT_INCLUDES
	#include <AsDefault.h>
#endif

void _INIT ProgramInit(void)
{
	fb_motor.Tm = 0.05;
	fb_motor.ke = 0.05;
	fb_motor.integrator.dt = 0.01;
	fb_motor.integrator.out = 0.0;
	fb_motor.phi = 0.0;

	fb_motor_2.Tm = 0.05;
	fb_motor_2.ke = 0.05;
	fb_motor_2.integrator.dt = 0.01;
	fb_motor_2.integrator.out = 0.0;
	fb_motor_2.phi = 0.0;

	REAL Tzh = 0.02;

	fb_regulator.k_p = fb_motor.ke * fb_motor.Tm / Tzh;
	fb_regulator.k_i = fb_motor.ke / Tzh;
	fb_regulator.max_abs_value = 24.0;
	fb_regulator.max_abs_value = 24.0;
	fb_regulator.integrator.dt = 0.01;
	fb_regulator.integrator.out = 0.0;

	Counter = 0;
	Speed = 0;
	Enable = 0;
}

void _CYCLIC ProgramCyclic(void)
{
	Counter ++;
	if (Counter > 100)
	{
		Speed = 90.0;
		Enable = 1;
```

```
	}
	if (Counter > 250)
	{
		Speed = 0.0;
	}
	fb_motor_2.u = Enable ? (Speed * fb_motor_2.ke) : 0.0;
	FB_Motor(&fb_motor_2);

	if (Enable)
	{
		fb_regulator.e = Speed - fb_motor.w;
		FB_Regulator(&fb_regulator);

		fb_motor.u = fb_regulator.u;
		FB_Motor(&fb_motor);
	}
	else
	{
		fb_motor.u = 0.0;
		FB_Motor(&fb_motor);
	}
}
```